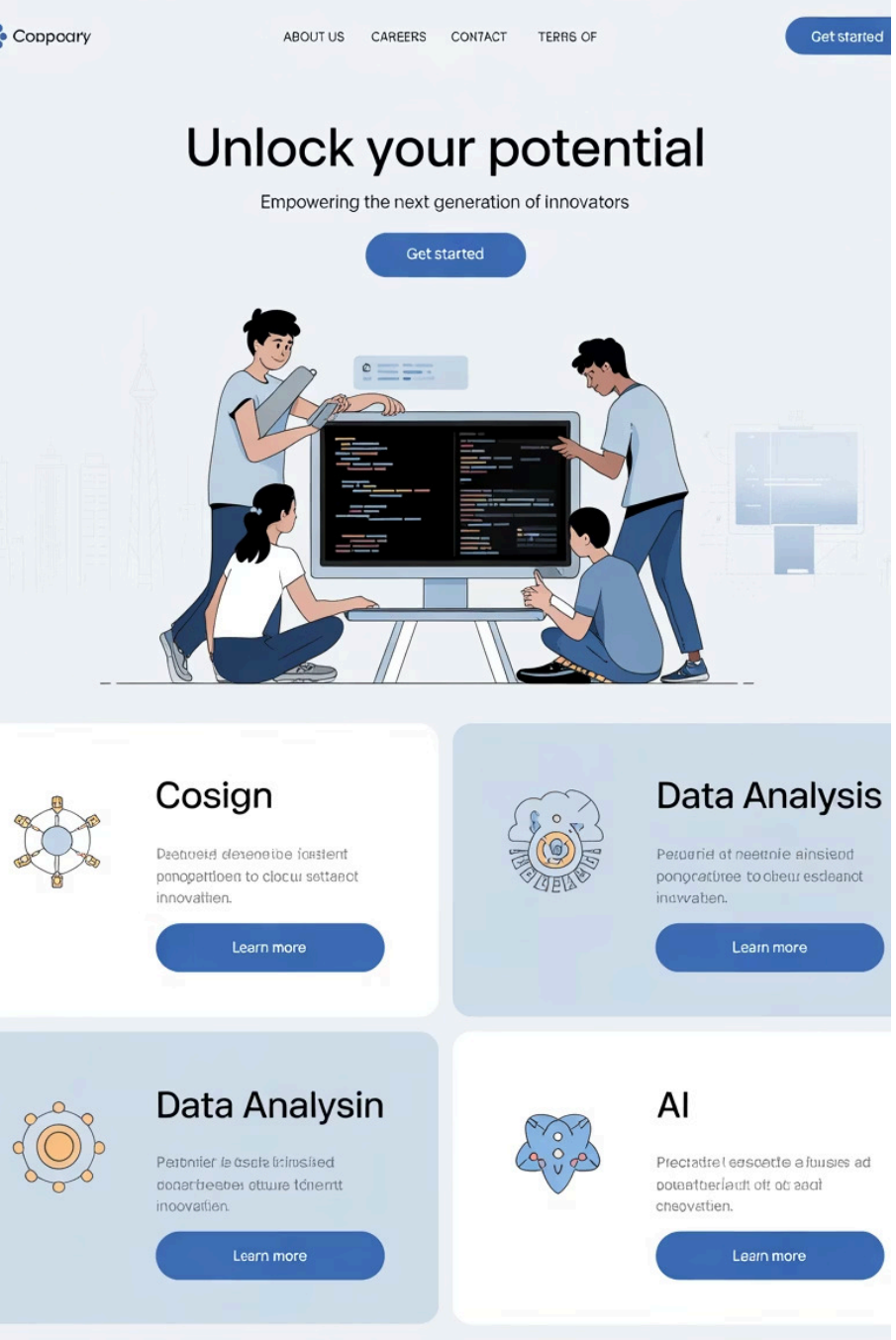


Guia Prático: Colaborando com Git e GitHub

Bem-vindos à disciplina de Controle de Versionamento! Neste curso, vocês aprenderão a usar o Git e GitHub para colaboração efetiva em projetos de software, habilidades essenciais para qualquer engenheiro de software.

Agenda do Curso

01

Introdução ao Git e GitHub

Compreensão dos conceitos básicos e configuração inicial das ferramentas.

02

Fluxo de Trabalho Colaborativo

Clonagem, branches, commits, e pull requests em ambiente real.

03

Resolução de Conflitos

Identificação e resolução de problemas de merge durante a colaboração.

04

Publicação Automática

Verificação da publicação das contribuições via GitHub Pages.

Repositório da Turma: **https://github.com/rlzambroti/turma-2025-b3**

Página Publicada: https://rlzambroti.github.io/turma-2025-b3/

O Desafio do Projeto

Objetivo: Contribuir para um projeto colaborativo real

Vocês irão:

- Criar um portfólio pessoal em HTML/CSS
- Adicionar seu perfil ao arquivo da turma
- Publicar suas contribuições usando Git e GitHub
- Resolver conflitos de merge que surgirem
- Ver suas contribuições publicadas em tempo real

Este desafio simula o dia a dia de um desenvolvedor em equipe, proporcionando experiência prática essencial para sua carreira!

Pré-requisitos para o Projeto

1

Git Instalado e Configurado

Certifique-se de que o Git está corretamente instalado em seu computador, com as configurações básicas:

```
git config --global user.name "Seu Nome"
git config --global user.email "seu.email@exemplo.com"
```

2

Conta Ativa no GitHub

Você precisa ter uma conta no GitHub e estar adicionado como colaborador ao repositório do Professor Robson Zambroti.

3

Editor de Código

Um bom editor de texto ou IDE como VS Code, Sublime Text ou Atom, com suporte para HTML, CSS e JavaScript.

4

Conexão com a Internet

Essencial para interagir com o GitHub e publicar suas alterações.

Fluxo de Trabalho Completo

Clone

Copie o repositório para sua máquina local

Branch

Crie sua própria branch de trabalho

Código

Desenvolva seu portfólio e adicione seu perfil

Commit

Registre suas alterações

Push

Envie para o GitHub

Pull Request

Solicite a integração das suas mudanças

Passo 1: Clonando o Repositório

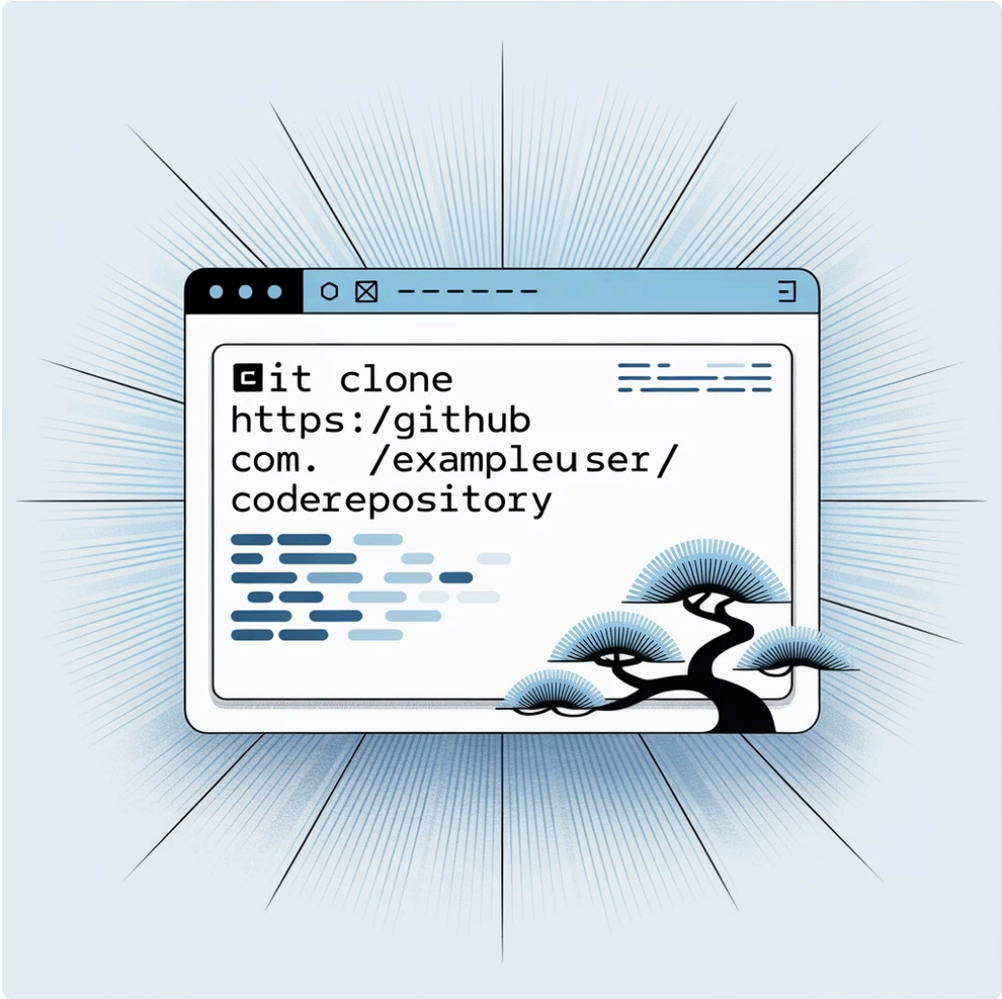
Para começar, você precisa obter uma cópia local do projeto. Abra seu terminal e execute:

```
git clone https://github.com/rlzambroti/turma-2025-b3.git
```

Em seguida, entre no diretório do projeto:

```
cd turma-2025-b3
```

Agora você tem uma cópia completa do repositório na sua máquina, incluindo todo o histórico de alterações!



A clonagem é o primeiro passo essencial para qualquer colaboração em Git. Ela baixa todo o projeto, incluindo todas as branches e todo o histórico de alterações.

Passo 2: Criando sua Branch

É fundamental criar sua própria branch para trabalhar isoladamente:

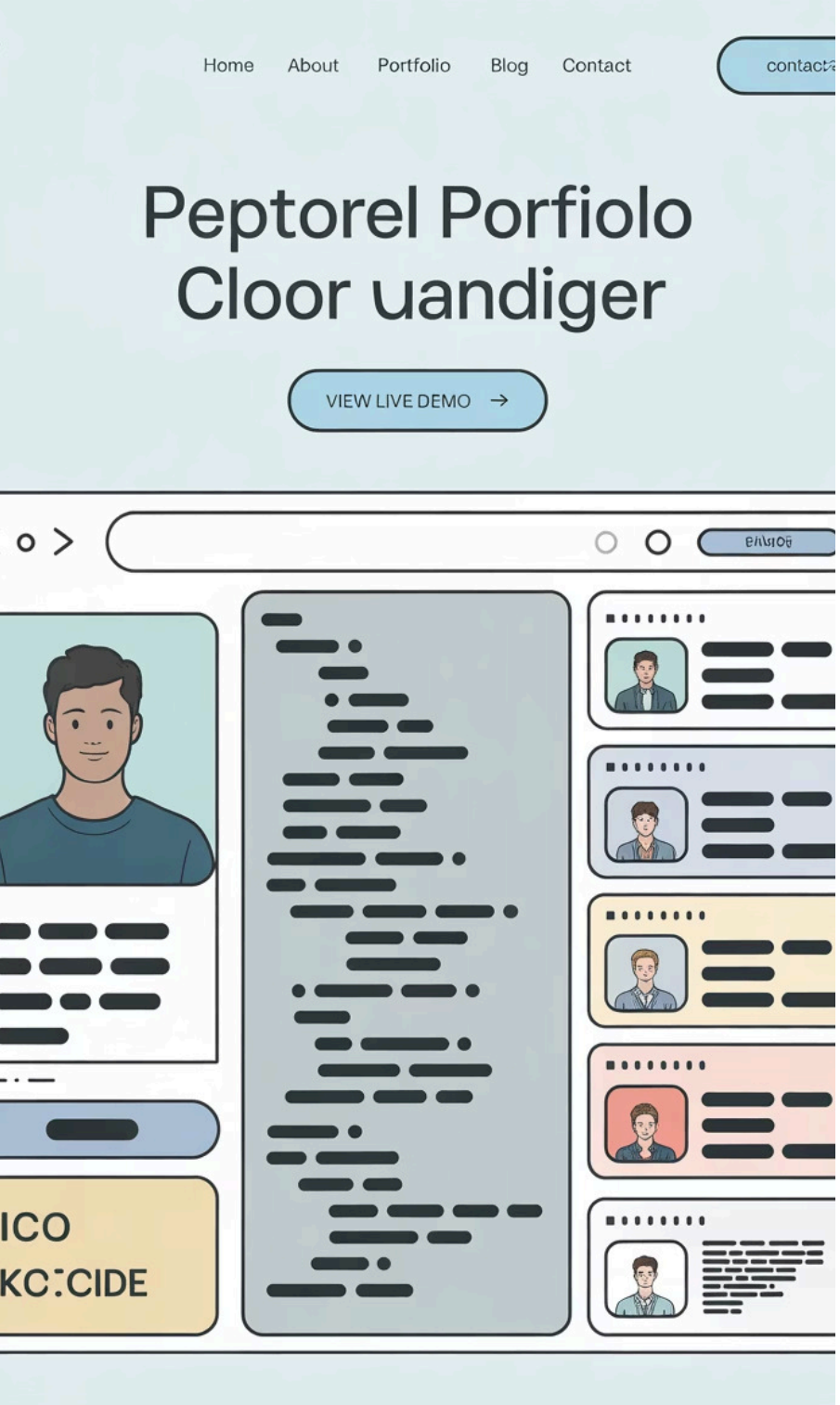
```
git switch -c perfil-SUAS_INICIAIS-SUA_MATRICULA
```

Exemplo: Se você é Ana Silva com matrícula 12345, use:

```
git switch -c perfil-as-12345
```

> **Importante:** O nome da sua branch deve seguir o padrão `perfil-INICIAIS-MATRICULA` para fácil identificação!

Trabalhar em branches separadas permite que todos desenvolvam simultaneamente sem interferir no trabalho dos outros.



Passo 3: Criando seu Portfólio

Crie seu arquivo HTML

Na pasta raiz do projeto, crie um arquivo HTML seguindo o padrão de nomenclatura: `SUAS_INICIAIS-SUA_MATRICULA.html`

Exemplo: `rs-2025001.html` para Robson Zambroti, matrícula 2025001

Personalize o conteúdo

Use o arquivo `rlz-99999.html` como modelo, mas personalize com suas informações pessoais, interesses e habilidades.

Crie os recursos

Para arquivos CSS, JavaScript ou imagens, use sua matrícula como sufixo antes da extensão:

- CSS: `style-SUA_MATRICULA.css`
- JavaScript: `script-SUA_MATRICULA.js`
- Imagens: minhafoto-SUA_MATRICULA.jpg

Passo 4: Atualizando o arquivo turma.md

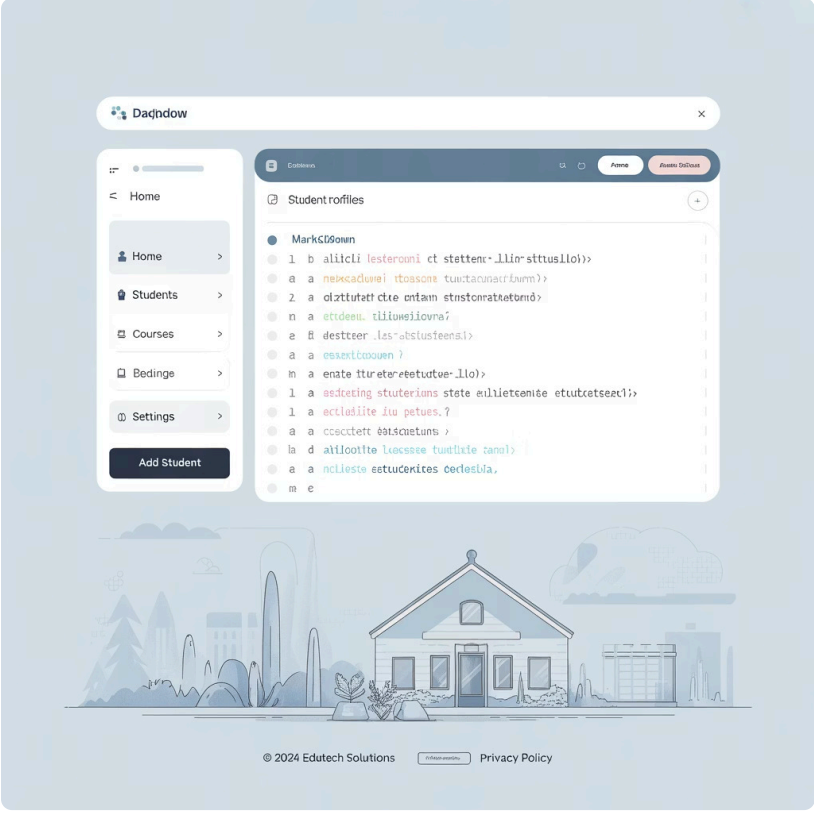
Abra o arquivo `turma.md` no seu editor e adicione sua entrada no **final da lista**, seguindo este formato:

```
### Nome: Seu Nome Completo

**Área de Interesse:** Sua área de interesse
**Porquê:** Breve explicação do seu interesse
**Meu Portfólio:** [Link para meu portfólio](SEU_ARQUIVO_PORTFOLIO.html)
```

Substitua `SEU_ARQUIVO_PORTFOLIO.html` pelo nome do arquivo HTML que você criou no Passo 3.

Exemplo: `[Link para meu portfólio](rs-2025001.html)`



O arquivo `turma.md` funcionará como um índice coletivo para todos os portfólios da turma.

Passos 5 e 6: Staging e Commit

Preparando suas alterações (Staging)

Verifique o status para ver os arquivos modificados:

```
git status
```

Adicione seus arquivos à área de staging:

```
git add turma.md SEU_ARQUIVO_PORTFOLIO.html
git add SEUS_ARQUIVOS_CSS_OU_IMAGENS.ext
```

Exemplo:

```
git add turma.md rs-2025001.html
git add style-2025001.css minhafoto-2025001.jpg
```

Registrando suas alterações (Commit)

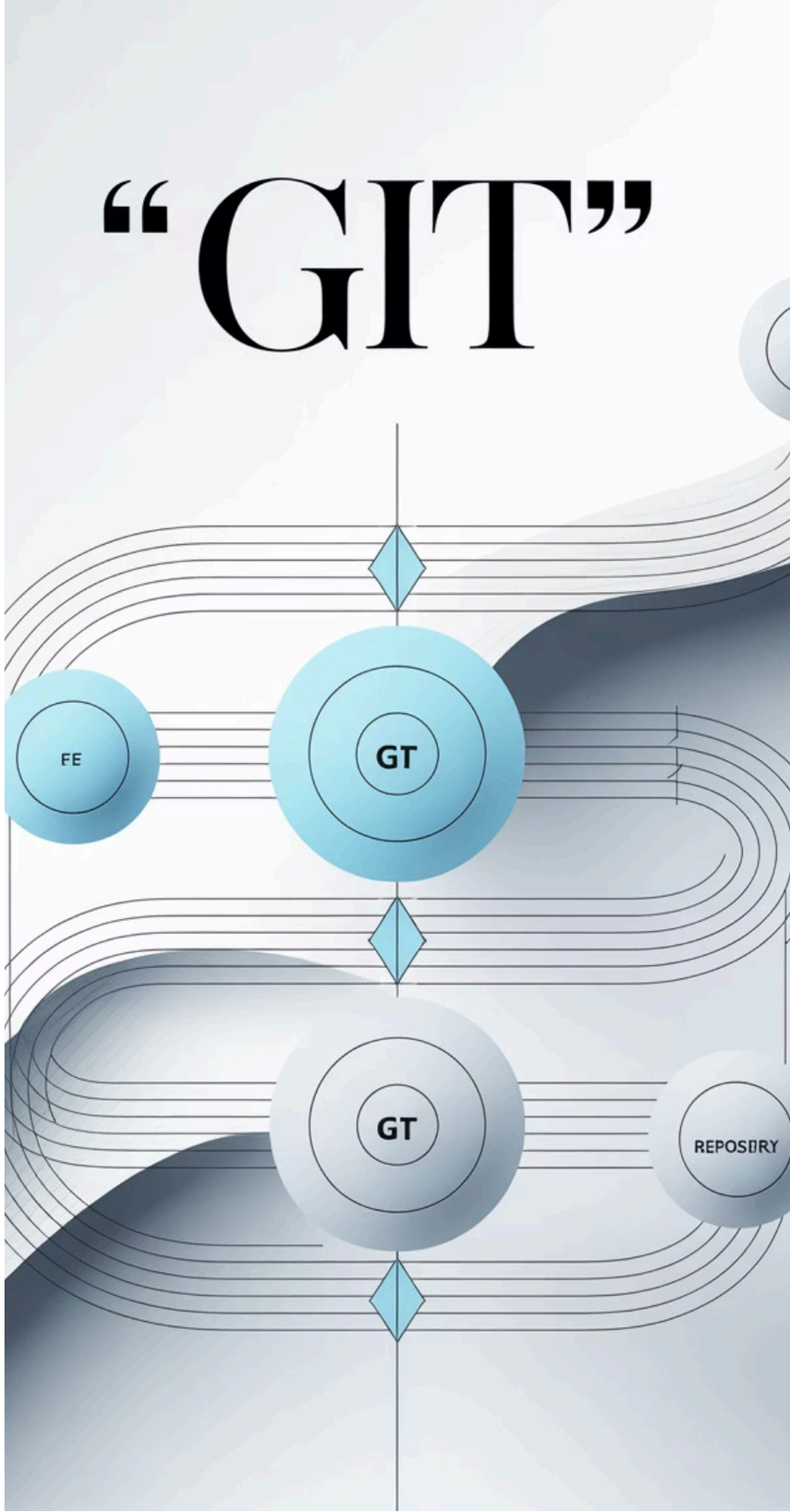
Crie um commit com uma mensagem descritiva:

```
git commit -m "feat: Adiciona perfil de [Seu Nome] e portfólio"
```

Exemplo:

```
git commit -m "feat: Adiciona perfil de Robson Zambroti e portfólio"
```

Use prefixos como "feat:", "fix:", "docs:" para categorizar seus commits.



Passo 7: Push para o GitHub

Envie suas alterações para o GitHub com o comando:

```
git push origin perfil-SUAS_INICIAIS-SUA_MATRICULA
```

Exemplo:

```
git push origin perfil-rs-2025001
```

Se for a primeira vez que você envia para essa branch, o Git pode sugerir um comando ligeiramente diferente:

```
git push --set-upstream origin perfil-SUAS_INICIAIS-SUA_MATRICULA
```

Basta copiar e colar o comando sugerido pelo terminal.

O comando push envia todas as alterações commitadas na sua branch local para o repositório remoto no GitHub.

Passo 8: Criando o Pull Request

Após o push, vá para o repositório no GitHub: https://github.com/rlzambroti/turma-2025-b3

1. Inicie o Pull Request

Clique no botão "Compare & pull request" que aparecerá após o push, ou vá manualmente à seção Pull Requests e clique em "New pull request".

2. Configure as branches

Certifique-se que:

- **base:** está definida para `main`
- **compare:** está definida para sua branch (`perfil-SUAS_INICIAIS-SUA_MATRICULA`)

3. Descreva seu PR

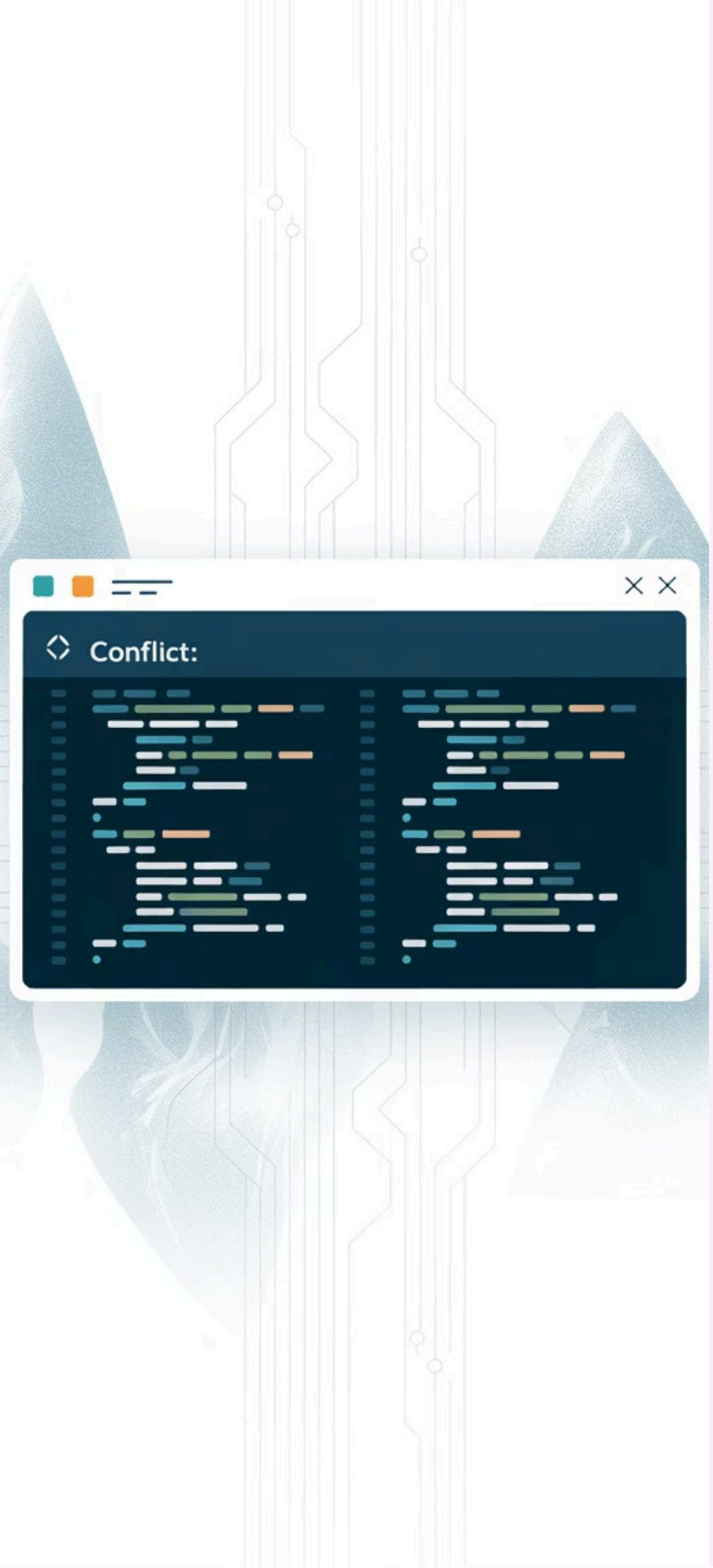
Dê um título claro e descritivo, como:

`feat: Meu Perfil e Portfólio - [Seu Nome Completo]`

Adicione uma descrição concisa do que você fez.

4. Finalize

Clique em "Create pull request" para submeter sua contribuição para revisão.



Passo 9: Resolução de Conflitos

⚠ **ATENÇÃO! Este é o ponto mais desafiador do processo!**

Como muitos alunos estarão editando o mesmo arquivo (`turma.md`), é provável que ocorram conflitos de merge. Este é um processo normal em desenvolvimento colaborativo!

1. Sincronize sua branch main

```
git switch main
git pull origin main
```

2. Volte para sua branch e faça o merge

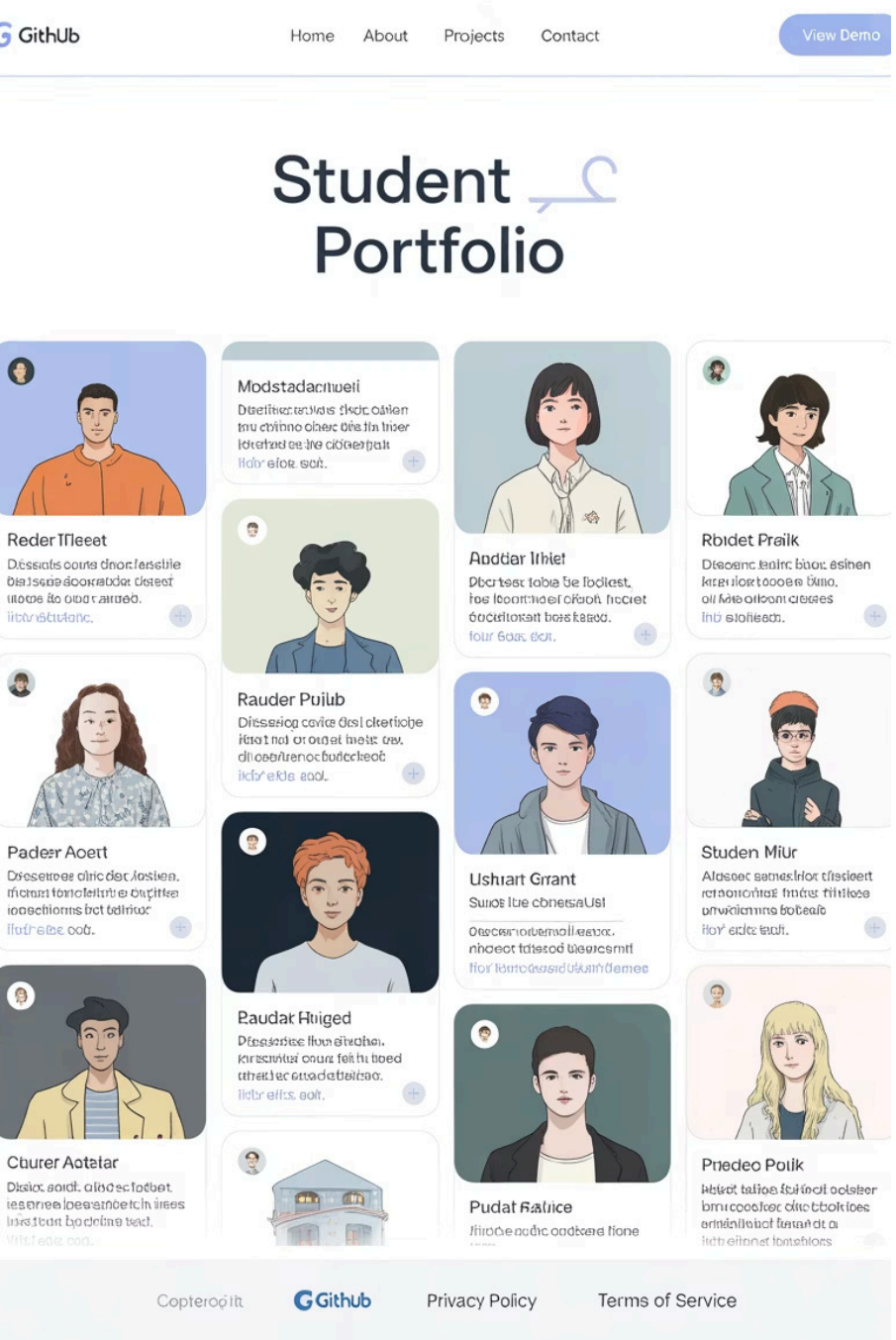
```
git switch perfil-SUAS_INICIAIS-SUA_MATRICULA
git merge main
```

3. Resolva os conflitos manualmente

Abra `turma.md` no seu editor, localize os marcadores `<<<<<<<`, `=======`, `>>>>>>>` e combine as versões, mantendo sua entrada E as entradas dos seus colegas.

4. Registre a resolução

```
git add turma.md
git commit -m "fix: Resolve conflito ao adicionar meu perfil"
git push origin perfil-SUAS_INICIAIS-SUA_MATRICULA
```



Passos 10 e 11: Publicação Final

Revisão e Merge

Após resolver os conflitos, seu Pull Request estará pronto para revisão.

1. O Professor Robson revisará seu PR
2. Se aprovado, ele mesclará suas alterações na branch main
3. O GitHub Actions automaticamente publicará as alterações

Este processo simula o ambiente real de desenvolvimento, onde cada contribuição passa por revisão antes de ser integrada.

Verificação da Publicação

Após o merge, suas alterações serão publicadas no GitHub Pages:

1. Acesse: https://rlzambroti.github.io/turma-2025-b3/
2. Clique em "Lista da Turma (turma.md)"
3. Localize seu nome e clique no link do seu portfólio

A publicação pode levar de 30 segundos a 2 minutos para ser atualizada.

Dicas e Boas Práticas

Use git status frequentemente

Este comando mostra o estado atual do seu repositório, quais arquivos foram modificados e quais estão prontos para commit.

Sincronize constantemente

Antes de começar a trabalhar, sempre atualize sua branch main e faça merge na sua branch de trabalho para minimizar conflitos.

Escreva boas mensagens de commit

Mensagens claras e concisas facilitam o entendimento do histórico do projeto. Use prefixos como "feat:", "fix:", "docs:".

Peça ajuda quando necessário

Se você ficar preso em algum passo ou tiver dificuldades com conflitos, não hesite em pedir ajuda ao professor ou colegas.

Este projeto não é apenas sobre completar uma tarefa, mas sobre aprender habilidades que vocês usarão durante toda a carreira como engenheiros de software!